

Misi 2 - DESAIN FISIKAL DAN DEVELOPMENT

Step 1: Physical Database Design

Desain fisikaal ini mengimplementasikan Model Dimensional Unit Keuangan ke dalam lingkungan database SQL Server, yang mencakup definisi skema, tabel, *Foreign Key (FK)*, hingga strategi *indexing* dan *partitioning*.

1. Step 1: Physical Database Design

a) Database setup

```
-- =====  
-- 01_Create_Database.sql  
-- Create Data Warehouse database for Keuangan  
-- =====  
CREATE DATABASE DM_Keuangan_DW  
ON PRIMARY  
(  
    NAME = N'DM_Keuangan_DW_Data',  
    FILENAME = N'D:\Data\DM_Keuangan_DW_Data.mdf',  
    SIZE = 1GB,  
    MAXSIZE = UNLIMITED,  
    FILEGROWTH = 256MB  
)  
LOG ON  
(  
    NAME = N'DM_Keuangan_DW_Log',  
    FILENAME = N'E:\Logs\DM_Keuangan_DW_Log.ldf',  
    SIZE = 256MB,  
    MAXSIZE = 2GB,  
    FILEGROWTH = 64MB  
);  
GO  
  
USE DM_Keuangan_DW;  
GO
```

b) Create Dimension Tables

```
USE DM_Keuangan_DW;
```

GO

--

=====

=====

-- 2. PEMBUATAN TABEL DIMENSI (DENGAN SCD TIPE 2)

--

=====

=====

-- 2.1 Tabel Dimensi Pos Akun (Dim_Pos_Akun) - SCD Tipe 2

CREATE TABLE dbo.Dim_Pos_Akun (

 ID_Akun INT PRIMARY KEY IDENTITY(1,1) NOT NULL,

 Kode_Akun VARCHAR(20) NOT NULL, -- Natural Key (Tanpa

UNIQUE)

 Nama_Akun VARCHAR(100) NOT NULL,

 Tipe_Akun VARCHAR(50) NOT NULL,

 Grup_Akun VARCHAR(50) NOT NULL,

 Start_Date DATE NOT NULL DEFAULT '2021-01-01', -- Kolom

SCD Tipe 2

 End_Date DATE NULL, -- Kolom SCD Tipe 2

 Is_Current_Flag CHAR(1) NOT NULL DEFAULT 'Y' -- Kolom

SCD Tipe 2

);

GO

-- 2.2 Tabel Dimensi Unit Organisasi (Dim_Unit_Org) - SCD Tipe 2

CREATE TABLE dbo.Dim_Unit_Org (

 ID_Unit INT PRIMARY KEY IDENTITY(1,1) NOT NULL,

 Kode_Unit VARCHAR(10) NOT NULL, -- Natural Key (Tanpa

UNIQUE)

 Nama_Unit VARCHAR(100) NOT NULL,

 Nama_Fakultas VARCHAR(100) NULL,

 Tipe_Unit VARCHAR(50) NOT NULL,

 Start_Date DATE NOT NULL DEFAULT '2021-01-01', -- Kolom

SCD Tipe 2

 End_Date DATE NULL, -- Kolom SCD Tipe 2

 Is_Current_Flag CHAR(1) NOT NULL DEFAULT 'Y' -- Kolom

SCD Tipe 2

);

GO

-- 2.3 Tabel Dimensi Vendor/Pemasok (Dim_Vendor) - SCD Tipe 2

```
CREATE TABLE dbo.Dim_Vendor (  
    ID_Vendor      INT          PRIMARY KEY IDENTITY(1,1) NOT NULL,  
    Kode_Vendor    VARCHAR(20)  NOT NULL, -- Natural Key (Tanpa  
    UNIQUE)  
    Nama_Vendor    VARCHAR(100) NOT NULL,  
    Kategori_Vendor VARCHAR(50) NULL,  
    Kota           VARCHAR(100) NULL,  
    Start_Date     DATE         NOT NULL DEFAULT '2021-01-01', -- Kolom  
    SCD Tipe 2  
    End_Date       DATE         NULL, -- Kolom SCD Tipe 2  
    Is_Current_Flag CHAR(1)     NOT NULL DEFAULT 'Y' -- Kolom  
    SCD Tipe 2  
);  
GO
```

-- 2.4 Tabel Dimensi Sumber Dana (Dim_Sumber_Dana) - SCD Tipe 2

```
CREATE TABLE dbo.Dim_Sumber_Dana (  
    ID_Sumber      INT          PRIMARY KEY IDENTITY(1,1) NOT NULL,  
    Kode_Sumber    VARCHAR(10)  NOT NULL, -- Natural Key (Tanpa  
    UNIQUE)  
    Deskripsi_Sumber VARCHAR(100) NOT NULL,  
    Tipe_Pendanaan VARCHAR(20)  NOT NULL,  
    Start_Date     DATE         NOT NULL DEFAULT '2021-01-01', -- Kolom  
    SCD Tipe 2  
    End_Date       DATE         NULL, -- Kolom SCD Tipe 2  
    Is_Current_Flag CHAR(1)     NOT NULL DEFAULT 'Y' -- Kolom  
    SCD Tipe 2  
);  
GO
```

-- 2.5 Tabel Dimensi Waktu (Dim_Waktu) - SCD Tipe 0

```
CREATE TABLE dbo.Dim_Waktu (  
    ID_Waktu      INT          PRIMARY KEY NOT NULL,  
    Tanggal       DATE         NOT NULL UNIQUE,  
    Tahun         INT          NOT NULL,  
    Kuartal       VARCHAR(5)   NOT NULL,  
    Bulan         VARCHAR(15)  NOT NULL,
```

```

    Hari_Kerja_Flag BIT NOT NULL DEFAULT 1
);
GO

```

c) Create Fact Tables

```

--
=====
-- 3. PEMBUATAN TABEL FAKTA
--
=====

-- 3.1 Tabel Fact_Transaksi
CREATE TABLE dbo.Fact_Transaksi (
    ID_Fakta_Transaksi BIGINT PRIMARY KEY IDENTITY(1,1)
NOT NULL,
    -- Foreign Keys
    ID_Waktu INT NOT NULL,
    ID_Akun INT NOT NULL,
    ID_Unit INT NOT NULL,
    ID_Vendor INT NULL,
    ID_Sumber INT NOT NULL,
    -- Degenerate Dimension
    ID_Transaksi_Operasional VARCHAR(50) NULL,
    -- Measures
    Nilai_Debit DECIMAL(15,2) NOT NULL,
    Nilai_Kredit DECIMAL(15,2) NOT NULL,
    -- Metadata
    SourceSystemVar VARCHAR(50) NOT NULL DEFAULT
'SourceSystem',
    LoadDate DATETIME DEFAULT GETDATE(),

    -- Foreign Key Constraints
    CONSTRAINT FK_FT_Waktu FOREIGN KEY (ID_Waktu)
REFERENCES dbo.Dim_Waktu(ID_Waktu),
    CONSTRAINT FK_FT_Akun FOREIGN KEY (ID_Akun) REFERENCES
dbo.Dim_Pos_Akun(ID_Akun),
    CONSTRAINT FK_FT_Unit FOREIGN KEY (ID_Unit) REFERENCES
dbo.Dim_Unit_Org(ID_Unit),
    CONSTRAINT FK_FT_Vendor FOREIGN KEY (ID_Vendor)
REFERENCES dbo.Dim_Vendor(ID_Vendor),
    CONSTRAINT FK_FT_Sumber FOREIGN KEY (ID_Sumber)

```

```

REFERENCES dbo.Dim_Sumber_Dana(ID_Sumber)
);
GO

-- 3.2 Tabel Fact_Anggaran
CREATE TABLE dbo.Fact_Anggaran (
    ID_Fakta_Anggaran BIGINT PRIMARY KEY IDENTITY(1,1) NOT
NULL,
    -- Foreign Keys
    ID_Waktu INT NOT NULL,
    ID_Akun INT NOT NULL,
    ID_Unit INT NOT NULL,
    -- Measures
    Nilai_Anggaran DECIMAL(15,2) NOT NULL,
    Nilai_Realisasi DECIMAL(15,2) NOT NULL,
    Persen_Realisasi DECIMAL(5,2) NULL,
    -- Metadata
    LoadDate DATETIME DEFAULT GETDATE(),

    -- Foreign Key Constraints
    CONSTRAINT FK_FactAnggaran_Waktu FOREIGN KEY (ID_Waktu)
REFERENCES dbo.Dim_Waktu(ID_Waktu),
    CONSTRAINT FK_FactAnggaran_Akun FOREIGN KEY (ID_Akun)
REFERENCES dbo.Dim_Pos_Akun(ID_Akun),
    CONSTRAINT FK_FactAnggaran_Unit FOREIGN KEY (ID_Unit)
REFERENCES dbo.Dim_Unit_Org(ID_Unit)
);
GO

```

2. Step 2: Indexing Strategy

a) Non-Clustered Indexes

```

USE DM_Keuangan_DW;
GO

--
=====

-- 1. NON-CLUSTERED INDEXES pada DIMENSION TABLES (Optimasi
ETL & Filtering Atribut)
--

```

```
=====
-- Natural Key Index: Mempercepat pencarian Surrogate Key (SK) selama
proses ETL.
```

```
CREATE NONCLUSTERED INDEX IX_DimVendor_KodeVendor ON
dbo.Dim_Vendor (Kode_Vendor);
```

```
GO
```

```
CREATE NONCLUSTERED INDEX IX_DimAkun_KodeAkun ON
dbo.Dim_Pos_Akun (Kode_Akun);
```

```
GO
```

```
-- Index Atribut Dimensi: Mempercepat Filtering dan Grouping berdasarkan
hierarki/atribut.
```

```
-- Kebutuhan: Analisis Tipe Akun (misal: semua 'Beban' atau 'Pendapatan').
```

```
CREATE NONCLUSTERED INDEX IX_DimAkun_TipeGrup ON
dbo.Dim_Pos_Akun (Tipe_Akun, Grup_Akun);
```

```
GO
```

```
-- Kebutuhan: Filtering waktu (misal: WHERE Tahun=2024).
```

```
CREATE NONCLUSTERED INDEX IX_DimWaktu_TahunKuartal ON
dbo.Dim_Waktu (Tahun, Kuartal);
```

```
GO
```

```
--
```

```
=====
-- 2. COVERING INDEXES pada FACT TABLES (Optimasi KPI)
```

```
-- Dibuat pada Foreign Key (FK) dan menyertakan Measures (Nilai)
```

```
--
```

```
=====
-- Covering Index 1: Unit Performance (Mendukung query per Unit/Fakultas)
```

```
CREATE NONCLUSTERED INDEX IX_FactTransaksi_UnitCovering
ON dbo.Fact_Transaksi (ID_Unit)
```

```
INCLUDE (Nilai_Debit, Nilai_Kredit);
```

```
GO
```

```
-- Covering Index 2: Akun Performance (Mendukung query per Kategori Akun)
```

```
CREATE NONCLUSTERED INDEX IX_FactTransaksi_AkunCovering
ON dbo.Fact_Transaksi (ID_Akun)
```

```
INCLUDE (Nilai_Debit, Nilai_Kredit);
```

```
GO
```

```
-- Covering Index 3: Tren Unit Kerja (Mendukung query Waktu X Unit - KPI
Eksekutif)
```

```

CREATE NONCLUSTERED INDEX IX_FactTransaksi_TrenUnit
ON dbo.Fact_Transaksi (ID_Waktu, ID_Unit)
INCLUDE (Nilai_Debit, Nilai_Kredit);
GO

-- Covering Index 4: Kinerja Anggaran Unit (KPI Varian Anggaran per Unit)
CREATE NONCLUSTERED INDEX IX_FactAnggaran_UnitCovering
ON dbo.Fact_Anggaran (ID_Unit)
INCLUDE (Nilai_Anggaran, Nilai_Realisasi, Persen_Realisasi);
GO

-- Covering Index 5: Kinerja Anggaran Akun (KPI Varian Anggaran per Akun)
CREATE NONCLUSTERED INDEX IX_FactAnggaran_AkunCovering
ON dbo.Fact_Anggaran (ID_Akun)
INCLUDE (Nilai_Anggaran, Nilai_Realisasi, Persen_Realisasi);
GO

```

b) Columnstore Index (for large fact tables)

```

-- Columnstore Index untuk Fact_Transaksi
-- Meliputi semua Foreign Keys dan Measures (Nilai_Debit, Nilai_Kredit)
CREATE NONCLUSTERED COLUMNSTORE INDEX
NCCIX_Fact_Transaksi
ON dbo.Fact_Transaksi
(
    ID_Waktu,      -- Foreign Key
    ID_Akun,      -- Foreign Key
    ID_Unit,       -- Foreign Key
    ID_Vendor,     -- Foreign Key
    ID_Sumber,     -- Foreign Key
    Nilai_Debit,   -- Measure
    Nilai_Kredit   -- Measure
    -- ID_Transaksi_Operasional (Degenerate Dimension) biasanya tidak
    -- diindeks di CCI
    -- kecuali jika sering digunakan sebagai filter tunggal.
);
GO

USE DM_Keuangan_DW;
GO

-- Columnstore Index untuk Fact_Anggaran
-- Meliputi semua Foreign Keys dan Measures (Nilai_Anggaran,

```

```

Nilai_Realisasi)
CREATE NONCLUSTERED COLUMNSTORE INDEX
NCCIX_Fact_Anggaran
ON dbo.Fact_Anggaran
(
    ID_Waktu,      -- Foreign Key
    ID_Akun,      -- Foreign Key
    ID_Unit,       -- Foreign Key
    Nilai_Anggaran, -- Measure
    Nilai_Realisasi, -- Measure
    Persen_Realisasi -- Calculated Measure
);
GO

```

3. Step 3: Partitioning Strategy

```

USE DM_Keuangan_DW;
GO

-- =====
-- 1. Membuat FUNGSI PARTISI (Partition Function)
-- =====
CREATE PARTITION FUNCTION PF_Tahunn (INT)
AS RANGE RIGHT FOR VALUES
(
    20210101, -- Data < 2021 masuk Partisi 1
    20220101, -- Data 2021 masuk Partisi 2
    20230101, -- Data 2022 masuk Partisi 3
    20240101, -- Data 2023 masuk Partisi 4
    20250101 -- Data 2024 masuk Partisi 5 (Data >= 2025 masuk Partisi 6)
);
GO

-- =====
-- 2. Membuat SKEMA PARTISI (Partition Scheme)
-- =====
-- Mengarahkan semua potongan data tadi ke filegroup PRIMARY
CREATE PARTITION SCHEME PS_Tahunn
AS PARTITION PF_Tahunn
ALL TO ([PRIMARY]);
GO

```


4. Step 4 : ETL

Pada tahap ini dirancang arsitektur ETL (Extract, Transform, Load) yang digunakan untuk memindahkan data dari sumber ke dalam Data Warehouse Unit Keuangan. Karena data operasional asli tidak tersedia, proses ETL dilakukan menggunakan data dummy, namun alur ETL tetap mengikuti standar implementasi Data Warehouse.

1. ETL Architecture Design

a. Source to Staging: Extract raw data

Tahap Extract melakukan pengambilan data dari berbagai sumber dummy yang mensimulasikan proses bisnis unit keuangan, antara lain:

- Dummy data Transaksi Keuangan (penerimaan dan pengeluaran)
- Dummy data Anggaran (perencanaan dan alokasi)
- Dummy data Realisasi Anggaran
- Dummy data Vendor/Rekanan
- Dummy data Unit Organisasi/Satuan Kerja
- Dummy data Pos Akun/Chart of Account
- Dummy data Sumber Dana/Sumber Pembiayaan
- Dummy data Periode/Waktu Fiskal

Pada tahap ini, data belum mengalami transformasi, melainkan langsung disalin apa adanya ke tabel staging agar tetap terjaga kelengkapannya.

Tujuan tahap ini:

- Menyalin data mentah secara apa adanya
- Menyediakan area buffer agar proses ETL dapat diulang tanpa memengaruhi sumber
- Menghindari perubahan langsung pada data sumber
- Mempertahankan audit trail data original

b. Staging to Integration: Data cleansing dan transformation

Data mentah yang berada pada staging selanjutnya dibersihkan, distandarisasi, dan ditransformasikan ke dalam Data Warehouse Unit Keuangan. Proses yang dilakukan meliputi:

- Normalisasi teks menggunakan TRIM() dan UPPER() untuk nama vendor, unit organisasi, dan deskripsi
- Validasi dan standarisasi format tanggal transaksi dan periode anggaran

- Mapping kode seperti kode akun, jenis transaksi, sumber dana, dan status
- Standardisasi nama unit organisasi dan vendor
- Validasi konsistensi nilai nominal (tidak negatif untuk penerimaan, format mata uang)
- Lookup referensi untuk menghasilkan surrogate key pada tabel dimensi
- Penanganan missing value atau data tidak konsisten
- Kalkulasi variance antara anggaran dan realisasi
- Agregasi data transaksi per periode untuk summary fact table

Tujuan tahap ini:

- Menghasilkan data yang bersih, konsisten, dan siap di-load ke Data Warehouse
- Memastikan struktur data sesuai dengan desain dimensional
- Menjamin integritas referensial antara fact dan dimension
- Memvalidasi business rules keuangan (balance, posting rules)

c. Integration to Data Warehouse: Load ke fact dan dimension tables

Setelah data melalui tahap transformasi, data dimuat ke dalam tabel dimensi (dimension) dan tabel fakta (fact) sesuai dengan model Data Warehouse yang telah dirancang.

Tabel Dimensi yang Diloat:

- **Dim_Waktu** - Dimensi waktu dengan granularitas harian, mingguan, bulanan, triwulan, dan tahunan
- **Dim_Sumber_Dana** - Master sumber pembiayaan (APBN, APBD, PNPB, Hibah, dll)
- **Dim_Vendor** - Master vendor/rekanan/pihak ketiga
- **Dim_Unit_Org** - Master unit organisasi/satuan kerja
- **Dim_Pos_Akun** - Chart of Account dengan hierarki akun
- **Dim_Jenis_Transaksi** - Kategori transaksi (penerimaan, pengeluaran, transfer)
- **Dim_Status** - Status transaksi atau anggaran (draft, approved, posted, cancelled)
- **Dim_Metode_Pembayaran** - Cara pembayaran (transfer, tunai, cek, dll)

Tabel Fakta yang Diloat:

- **Fact_Transaksi** - Detail transaksi keuangan harian (transaksi level)
- **Fact_Anggaran** - Data perencanaan anggaran per periode dan akun

Proses loading dilakukan dengan urutan:

1. Load tabel **Dim_Waktu** terlebih dahulu sebagai dimensi master

2. Load tabel dimensi lainnya (Dim_Sumber_Dana, Dim_Vendor, Dim_Unit_Org, Dim_Pos_Akun, dll) untuk menghasilkan surrogate key (DimKey)
3. Load tabel fakta dengan melakukan lookup DimKey dari dimensi terkait
4. Validasi integritas referensial dan business rules
5. Update statistik dan index untuk optimasi query

Tujuan tahap ini:

- Mengisi struktur Data Warehouse secara konsisten
- Mendukung kebutuhan analisis keuangan dan pelaporan melalui skema star schema
- Memungkinkan analisis multi-dimensi (OLAP) untuk budget vs actual, trend analysis, vendor spending
- Menyediakan single source of truth untuk data keuangan organisasi

2. Creating Staging Tables

Pada tahap ini dibuat tabel-tabel staging sebagai tempat penyimpanan data mentah (raw data) hasil proses extract dari berbagai sumber dummy unit keuangan. Tabel staging berfungsi sebagai area penampung sementara sebelum dilakukan proses cleansing, transformasi, dan loading ke dalam Data Warehouse.

Seluruh tabel staging ditempatkan pada schema **stg** untuk memisahkan area staging dari schema lainnya dan memudahkan manajemen data warehouse.

Berikut adalah daftar tabel staging yang akan dibuat:

Tabel Staging:

1. **stg_transaksi** - Staging untuk transaksi keuangan harian
2. **stg_anggaran** - Staging untuk data perencanaan anggaran
3. **stg_realisasi** - Staging untuk realisasi anggaran
4. **stg_vendor** - Staging untuk master vendor
5. **stg_unit_org** - Staging untuk master unit organisasi
6. **stg_pos_akun** - Staging untuk chart of account
7. **stg_sumber_dana** - Staging untuk sumber pembiayaan

Berikut adalah SQL script yang digunakan untuk membuat tabel-tabel staging:

```
CREATE SCHEMA stg;  
GO
```

--

=====

-- 4. PEMBUATAN TABEL STAGING

=====

--

-- Staging Master Akun

```
CREATE TABLE stg.Akun (  
    Kode_Akun    VARCHAR(255),  
    Nama_Akun    VARCHAR(255),  
    Tipe_Akun    VARCHAR(255),  
    Grup_Akun    VARCHAR(255)  
);  
GO
```

-- Staging Master Unit Organisasi

```
CREATE TABLE stg.Unit_Org (  
    Kode_Unit    VARCHAR(255),  
    Nama_Unit    VARCHAR(255),  
    Nama_Fakultas VARCHAR(255),  
    Tipe_Unit    VARCHAR(255)  
);  
GO
```

-- Staging Master Vendor/Pemasok

```
CREATE TABLE stg.Vendor (  
    Kode_Vendor  VARCHAR(255),  
    Nama_Vendor  VARCHAR(255),  
    Kategori_Vendor VARCHAR(255),  
    Kota         VARCHAR(255)  
);  
GO
```

-- Staging Master Sumber Dana

```
CREATE TABLE stg.Sumber_Dana (  
    Kode_Sumber  VARCHAR(255),  
    Deskripsi_Sumber VARCHAR(255),  
    Tipe_Pendanaan VARCHAR(255)  
);  
GO
```

-- Staging Transaksi Finansial

```
CREATE TABLE stg.Transaksi (  
    Tanggal_Transaksi VARCHAR(255),
```

```

Kode_Akun_NK      VARCHAR(255),
Kode_Unit_NK       VARCHAR(255),
Kode_Vendor_NK     VARCHAR(255),
Kode_Sumber_NK     VARCHAR(255),
ID_Transaksi_Operasional VARCHAR(255),
Nilai_Debit        VARCHAR(255),
Nilai_Kredit       VARCHAR(255)
);
GO

-- Staging Anggaran Realisasi
CREATE TABLE stg.Anggaran (
  Periode_Anggaran  VARCHAR(255),
  Kode_Akun_NK     VARCHAR(255),
  Kode_Unit_NK      VARCHAR(255),
  Nilai_Anggaran_Target VARCHAR(255),
  Nilai_Realisasi_Aktual VARCHAR(255)
);
GO

```

a) ETL Mapping Document
- Mapping Dim_Pos_Akun

Kolom Sumber (Staging)	Kolom Tujuan (Dimensi)	Tipe Data Tujuan	Logika / Transformasi
(System Generated)	ID_Akun	INT	<i>Auto Increment / Identity</i>
Kode_Akun	Kode_Akun	VARCHAR(20)	Langsung (Direct Move)
Nama_Akun	Nama_Akun	VARCHAR(100)	Langsung
Tipe_Akun	Tipe_Akun	VARCHAR(50)	Langsung
Grup_Akun	Grup_Akun	VARCHAR(50)	Langsung
(Default Value)	Start_Date	DATE	Default '2021-01-01' (SCD Type 2)
(Default Value)	End_Date	DATE	Default NULL (SCD Type 2)
(Default Value)	Is_Current_Flag	CHAR(1)	Default 'Y' (SCD Type 2)

Sumber : Tabel stg.Akun
Tujuan : Tabel dbo.Dim_Pos_Akun

- Mapping Dim_Unit_Org

Kolom Sumber (Staging)	Kolom Tujuan (Dimensi)	Tipe Data Tujuan	Logika / Transformasi
(System Generated)	ID_Unit	INT	<i>Auto Increment / Identity</i>
Kode_Unit	Kode_Unit	VARCHAR(10)	Langsung
Nama_Unit	Nama_Unit	VARCHAR(100)	Langsung
Nama_Fakultas	Nama_Fakultas	VARCHAR(100)	Langsung
Tipe_Unit	Tipe_Unit	VARCHAR(50)	Langsung
(Default Value)	Start_Date	DATE	Default '2021-01-01'
(Default Value)	Is_Current_Flag	CHAR(1)	Default 'Y'

Sumber : Tabel stg.Unit_Org
Tujuan : Tabel dbo.Dim_Unit_Org

- Mapping Dim_Vendor

Kolom Sumber (Staging)	Kolom Tujuan (Dimensi)	Tipe Data Tujuan	Logika / Transformasi
(System Generated)	ID_Vendor	INT	<i>Auto Increment / Identity</i>
Kode_Vendor	Kode_Vendor	VARCHAR(20)	Langsung
Nama_Vendor	Nama_Vendor	VARCHAR(100)	Langsung
Kategori_Vendor	Kategori_Vendor	VARCHAR(50)	Langsung
Kota	Kota	VARCHAR(100)	Langsung
(Default Value)	Start_Date	DATE	Default '2021-01-01'

Sumber : Tabel stg.Vendor
Tujuan : Tabel dbo.Dim_Vendor

- Mapping Dim_Sumber_Dana

Kolom Sumber (Staging)	Kolom Tujuan (Dimensi)	Tipe Data Tujuan	Logika / Transformasi
(System Generated)	ID_Sumber	INT	<i>Auto Increment / Identity</i>
Kode_Sumber	Kode_Sumber	VARCHAR(10)	Langsung
Deskripsi_Sumber	Deskripsi_Sumber	VARCHAR(100)	Langsung
Tipe_Pendanaan	Tipe_Pendanaan	VARCHAR(20)	Langsung

Sumber : Tabel stg.Sumber_Dana

Tujuan : Tabel dbo.Dim_Sumber_Dana

- Mapping Fact_Transaksi

Kolom Sumber (Staging)	Kolom Tujuan (Fakta)	Tipe Data Tujuan	Logika / Transformasi
(System Generated)	ID_Fakta_Transaksi	BIGINT	<i>Auto Increment</i>
Tanggal_Transaksi	ID_Waktu	INT	Lookup ke Dim_Waktu (Convert Date to ID)
Kode_Akun_NK	ID_Akun	INT	Lookup ke Dim_Pos_Akun berdasarkan Kode
Kode_Unit_NK	ID_Unit	INT	Lookup ke Dim_Unit_Org berdasarkan Kode
Kode_Vendor_NK	ID_Vendor	INT	Lookup ke Dim_Vendor berdasarkan Kode

Kode_Sumber_NK	ID_Sumber	INT	Lookup ke Dim_Sumber_Dana berdasarkan Kode
ID_Transaksi_Operasional	ID_Transaksi_Operasional	VARCHAR(50)	Langsung
Nilai_Debit	Nilai_Debit	DECIMAL(15,2)	Convert String to Decimal
Nilai_Kredit	Nilai_Kredit	DECIMAL(15,2)	Convert String to Decimal
(System Date)	LoadDate	DATETIME	Default GETDATE()

Sumber : Tabel stg.Transaksi
Tujuan : Tabel dbo.Fact_Transaksi

- Mapping Fact_Anggaran

Kolom Sumber (Staging)	Kolom Tujuan (Fakta)	Tipe Data Tujuan	Logika / Transformasi
Periode_Anggaran	ID_Waktu	INT	Lookup Date Key (Mengambil Akhir Bulan)
Kode_Akun_NK	ID_Akun	INT	Lookup ke Dim_Pos_Akun
Kode_Unit_NK	ID_Unit	INT	Lookup ke Dim_Unit_Org
Nilai_Anggaran_Target	Nilai_Anggaran	DECIMAL(15,2)	Convert String to Decimal
Nilai_Realisasi_Aktual	Nilai_Realisasi	DECIMAL(15,2)	Convert String to Decimal
(Calculation)	Persen_Realisasi	DECIMAL(5,2)	(Realisasi / Anggaran) * 100

Sumber : Tabel stg.Anggaran

Tujuan : Tabel Fact_Anggaran

Step 5: ETL Implementation Menggunakan T-SQL Stored Procedures

```
--- Tabel Dim
CREATE PROCEDURE dbo.usp_Load_Dim_Pos_Akun
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Expire (Kedaluwarsa) baris lama yang mengalami perubahan atribut
    UPDATE d
    SET
        End_Date = DATEADD(day, -1, GETDATE()), -- Set End Date (sehari
        sebelum hari ini)
        Is_Current_Flag = 'N' -- Non-aktifkan flag saat ini
    FROM dbo.Dim_Pos_Akun d
    INNER JOIN stg.Pos_Akun s ON d.Kode_Akun = s.Kode_Akun
    WHERE d.Is_Current_Flag = 'Y' -- Hanya baris saat ini yang dicek
    AND (
        d>Nama_Akun <> s>Nama_Akun OR
        d.Tipe_Akun <> s.Tipe_Akun OR
        d.Grup_Akun <> s.Grup_Akun
    );

    -- 2. Insert baris baru (data baru atau perubahan data)
    INSERT INTO dbo.Dim_Pos_Akun (
        Kode_Akun, Nama_Akun, Tipe_Akun, Grup_Akun,
        Start_Date, End_Date, Is_Current_Flag
    )
    SELECT
        s.Kode_Akun,
        s>Nama_Akun,
        s.Tipe_Akun,
        s.Grup_Akun,
        GETDATE(), -- Start Date diatur ke tanggal proses
        NULL, -- End Date NULL (belum kadaluarsa)
        'Y' -- Flag Current diatur 'Y'
    FROM stg.Pos_Akun s
    -- Filter: Hanya insert jika tidak ada baris 'Current' yang sesuai
    WHERE NOT EXISTS (
        SELECT 1
        FROM dbo.Dim_Pos_Akun d
        WHERE d.Kode_Akun = s.Kode_Akun
```

```

        AND d.Is_Current_Flag = 'Y'
    );

END;
GO

CREATE PROCEDURE dbo.usp_Load_Dim_Unit_Org
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Expire (Kedaluwarsa) baris lama yang mengalami perubahan atribut
    UPDATE d
    SET
        End_Date = DATEADD(day, -1, GETDATE()),
        Is_Current_Flag = 'N'
    FROM dbo.Dim_Unit_Org d
    INNER JOIN stg.Unit_Org s ON d.Kode_Unit = s.Kode_Unit
    WHERE d.Is_Current_Flag = 'Y'
    AND (
        d>Nama_Unit <> s>Nama_Unit OR
        ISNULL(d>Nama_Fakultas, 'NULL') <> ISNULL(s>Nama_Fakultas,
'NULL') OR -- Tangani NULL
        d>Tipe_Unit <> s>Tipe_Unit
    );

    -- 2. Insert baris baru (data baru atau perubahan data)
    INSERT INTO dbo.Dim_Unit_Org (
        Kode_Unit, Nama_Unit, Nama_Fakultas, Tipe_Unit,
        Start_Date, End_Date, Is_Current_Flag
    )
    SELECT
        s.Kode_Unit,
        s>Nama_Unit,
        s>Nama_Fakultas,
        s>Tipe_Unit,
        GETDATE(),
        NULL,
        'Y'
    FROM stg.Unit_Org s
    WHERE NOT EXISTS (
        SELECT 1
        FROM dbo.Dim_Unit_Org d
        WHERE d.Kode_Unit = s.Kode_Unit
        AND d.Is_Current_Flag = 'Y'
    );

```

```
END;  
GO
```

```
CREATE PROCEDURE dbo.usp_Load_Dim_Vendor  
AS  
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    -- 1. Expire (Kedaluwarsa) baris lama yang mengalami perubahan atribut  
    UPDATE d
```

```
    SET
```

```
        End_Date = DATEADD(day, -1, GETDATE()),
```

```
        Is_Current_Flag = 'N'
```

```
    FROM dbo.Dim_Vendor d
```

```
    INNER JOIN stg.Vendor s ON d.Kode_Vendor = s.Kode_Vendor
```

```
    WHERE d.Is_Current_Flag = 'Y'
```

```
    AND (
```

```
        d>Nama_Vendor <> s>Nama_Vendor OR
```

```
        ISNULL(d.Kategori_Vendor, 'NULL') <> ISNULL(s.Kategori_Vendor,
```

```
'NULL') OR
```

```
        ISNULL(d.Kota, 'NULL') <> ISNULL(s.Kota, 'NULL')
```

```
    );
```

```
    -- 2. Insert baris baru (data baru atau perubahan data)
```

```
    INSERT INTO dbo.Dim_Vendor (
```

```
        Kode_Vendor, Nama_Vendor, Kategori_Vendor, Kota,
```

```
        Start_Date, End_Date, Is_Current_Flag
```

```
    )
```

```
    SELECT
```

```
        s.Kode_Vendor,
```

```
        s>Nama_Vendor,
```

```
        s.Kategori_Vendor,
```

```
        s.Kota,
```

```
        GETDATE(),
```

```
        NULL,
```

```
        'Y'
```

```
    FROM stg.Vendor s
```

```
    WHERE NOT EXISTS (
```

```
        SELECT 1
```

```
        FROM dbo.Dim_Vendor d
```

```
        WHERE d.Kode_Vendor = s.Kode_Vendor
```

```
        AND d.Is_Current_Flag = 'Y'
```

```
    );
```

```

END;
GO

CREATE PROCEDURE dbo.usp_Load_Dim_Sumber_Dana
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Expire (Kedaluwarsa) baris lama yang mengalami perubahan atribut
    UPDATE d
    SET
        End_Date = DATEADD(day, -1, GETDATE()),
        Is_Current_Flag = 'N'
    FROM dbo.Dim_Sumber_Dana d
    INNER JOIN stg.Sumber_Dana s ON d.Kode_Sumber = s.Kode_Sumber
    WHERE d.Is_Current_Flag = 'Y'
    AND (
        d.Deskripsi_Sumber <> s.Deskripsi_Sumber OR
        d.Tipe_Pendanaan <> s.Tipe_Pendanaan
    );

    -- 2. Insert baris baru (data baru atau perubahan data)
    INSERT INTO dbo.Dim_Sumber_Dana (
        Kode_Sumber, Deskripsi_Sumber, Tipe_Pendanaan,
        Start_Date, End_Date, Is_Current_Flag
    )
    SELECT
        s.Kode_Sumber,
        s.Deskripsi_Sumber,
        s.Tipe_Pendanaan,
        GETDATE(),
        NULL,
        'Y'
    FROM stg.Sumber_Dana s
    WHERE NOT EXISTS (
        SELECT 1
        FROM dbo.Dim_Sumber_Dana d
        WHERE d.Kode_Sumber = s.Kode_Sumber
        AND d.Is_Current_Flag = 'Y'
    );

END;
GO

CREATE PROCEDURE dbo.usp_Build_Dim_Waktu
    @StartDate DATE,

```

```

    @EndDate DATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Gunakan tabel sementara untuk membuat rentang tanggal
    WITH DateRange AS (
        SELECT @StartDate AS Tanggal
        UNION ALL
        SELECT DATEADD(day, 1, Tanggal)
        FROM DateRange
        WHERE Tanggal < @EndDate
    )
    INSERT INTO dbo.Dim_Waktu (
        ID_Waktu, Tanggal, Tahun, Kuartal, Bulan, Hari_Kerja_Flag
    )
    SELECT
        CAST(CONVERT(VARCHAR(8), dr.Tanggal, 112) AS INT) AS
ID_Waktu, -- YYYYMMDD
        dr.Tanggal,
        YEAR(dr.Tanggal) AS Tahun,
        'Q' + CAST(DATEPART(qq, dr.Tanggal) AS VARCHAR(1)) AS Kuartal,
        FORMAT(dr.Tanggal, 'MMMM', 'id-ID') AS Bulan, -- Menggunakan
format bahasa Indonesia
        CASE
            WHEN DATENAME(dw, dr.Tanggal) IN ('Saturday', 'Sunday') THEN 0
-- Asumsi hari kerja
            ELSE 1
        END AS Hari_Kerja_Flag
    FROM DateRange dr
    WHERE NOT EXISTS (
        SELECT 1 FROM dbo.Dim_Waktu dw WHERE dw.Tanggal = dr.Tanggal
    )
    OPTION (MAXRECURSION 0); -- Izinkan perulangan CTE tidak terbatas

END;
GO
-- Contoh Eksekusi:
-- EXEC dbo.usp_Build_Dim_Waktu '2020-01-01', '2030-12-31';

CREATE OR ALTER PROCEDURE dbo.usp_Load_Fact_Transaksii
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Transaksi (

```

```

ID_Waktu, ID_Akun, ID_Unit, ID_Vendor, ID_Sumber,
ID_Transaksi_Operasional,
Nilai_Debit, Nilai_Kredit,
SourceSystemVar
)
SELECT
-- Kunci Asing (Foreign Keys)
dw.ID_Waktu,
da.ID_Akun,
duo.ID_Unit,
dv.ID_Vendor,
dsd.ID_Sumber,

-- Degenerate Dimension & Measures
s.ID_Transaksi_Operasional,
CAST(s.Nilai_Debit AS DECIMAL(15,2)) AS Nilai_Debit, --
Transformasi tipe data
CAST(s.Nilai_Kredit AS DECIMAL(15,2)) AS Nilai_Kredit, --
Transformasi tipe data

-- Metadata
'Legacy_Source' AS SourceSystemVar
FROM
stg.Transaksi s

-- J O I N S: Menggunakan Natural Key (NK) dari tabel Staging (s)

-- 1. Dim_Waktu (Transformasi Tanggal)
INNER JOIN dbo.Dim_Waktu dw
ON CAST(s.Tanggal_Transaksi AS DATE) = dw.Tanggal

-- 2. Dim_Pos_Akun (Kode_Akun_NK)
INNER JOIN dbo.Dim_Pos_Akun da
ON s.Kode_Akun_NK = da.Kode_Akun
AND da.Is_Current_Flag = 'Y' -- Hanya ambil versi SCD Tipe 2 yang aktif

-- 3. Dim_Unit_Org (Kode_Unit_NK)
INNER JOIN dbo.Dim_Unit_Org duo
ON s.Kode_Unit_NK = duo.Kode_Unit
AND duo.Is_Current_Flag = 'Y'

-- 4. Dim_Vendor (Kode_Vendor_NK) - Menggunakan LEFT JOIN karena
ID_Vendor boleh NULL
LEFT JOIN dbo.Dim_Vendor dv
ON s.Kode_Vendor_NK = dv.Kode_Vendor
AND dv.Is_Current_Flag = 'Y'

```

```

-- 5. Dim_Sumber_Dana (Kode_Sumber_NK)
INNER JOIN dbo.Dim_Sumber_Dana dsd
    ON s.Kode_Sumber_NK = dsd.Kode_Sumber
    AND dsd.Is_Current_Flag = 'Y'

-- Pencegahan Duplikasi: Hanya masukkan transaksi yang belum ada
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Fact_Transaksi ft
    WHERE ft.ID_Transaksi_Operasional = s.ID_Transaksi_Operasional
    AND ft.ID_Waktu = dw.ID_Waktu
);

END;
GO

CREATE OR ALTER PROCEDURE dbo.usp_Load_Fact_Anggaran
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Anggaran (
        ID_Waktu, ID_Akun, ID_Unit,
        Nilai_Anggaran, Nilai_Realisasi, Persen_Realisasi
    )
    SELECT
        -- Kunci Asing
        dw.ID_Waktu,
        da.ID_Akun,
        du.ID_Unit,

        -- Measures (Transformasi tipe data)
        CAST(s.Nilai_Anggaran_Target AS DECIMAL(15,2)) AS
        Nilai_Anggaran,
        CAST(s.Nilai_Realisasi_Aktual AS DECIMAL(15,2)) AS Nilai_Realisasi,

        -- Calculated Measure
        CASE
            WHEN CAST(s.Nilai_Anggaran_Target AS DECIMAL(15,2)) > 0
            THEN (CAST(s.Nilai_Realisasi_Aktual AS DECIMAL(15,2)) /
            CAST(s.Nilai_Anggaran_Target AS DECIMAL(15,2))) * 100.0
            ELSE 0.00
        END AS Persen_Realisasi
    FROM
        stg.Anggaran s

```

```

-- J O I N S

-- 1. Dim_Waktu: Menggunakan s.ID_Waktu_NK (Asumsi ini adalah
Tanggal/ID Waktu)
INNER JOIN dbo.Dim_Waktu dw
    ON CAST(s.ID_Waktu_NK AS INT) = dw.ID_Waktu -- Join
menggunakan ID Waktu/DateKey

-- 2. Dim_Pos_Akun: Menggunakan s.ID_Akun_NK
INNER JOIN dbo.Dim_Pos_Akun da
    ON s.ID_Akun_NK = da.Kode_Akun
    AND da.Is_Current_Flag = 'Y'

-- 3. Dim_Unit_Org: Menggunakan s.ID_Unit_NK
INNER JOIN dbo.Dim_Unit_Org duo
    ON s.ID_Unit_NK = duo.Kode_Unit
    AND duo.Is_Current_Flag = 'Y'

-- Pencegahan Duplikasi
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Fact_Anggaran fa
    WHERE fa.ID_Waktu = dw.ID_Waktu
        AND fa.ID_Akun = da.ID_Akun
        AND fa.ID_Unit = duo.ID_Unit
);

END;
GO

```

Step 6 : Data Quality Assurance

```

USE DM_Keuangan_DW;
GO

PRINT '=== LAPORAN DATA QUALITY CHECK ===';

-- Cek 1: Pastikan tidak ada Transaksi yang "Anak Hilang" (Tidak punya
Waktu/Akun/Unit)
SELECT 'Data Transaksi Yg Tidak Punya Tanggal' AS Isu, COUNT(*) AS
Jumlah
FROM dbo.Fact_Transaksi WHERE ID_Waktu IS NULL

```



```

UNION ALL
SELECT 'Data Transaksi Yg Tidak Punya Akun', COUNT(*)
FROM dbo.Fact_Transaksi WHERE ID_Akun IS NULL
UNION ALL
SELECT 'Data Transaksi Yg Tidak Punya Unit', COUNT(*)
FROM dbo.Fact_Transaksi WHERE ID_Unit IS NULL;

-- Cek 2: Pastikan tidak ada Duplikasi di Dimensi Utama (Contoh: Akun)
SELECT Kode_Akun, COUNT(*) AS Jumlah_Duplikat
FROM dbo.Dim_Pos_Akun
GROUP BY Kode_Akun
HAVING COUNT(*) > 1;

-- Cek 3: Validasi Logika Bisnis (Debit/Kredit tidak boleh negatif aneh-aneh)
SELECT 'Transaksi Nilai Negatif' AS Isu, COUNT(*) AS Jumlah
FROM dbo.Fact_Transaksi
WHERE Nilai_Debit < 0 OR Nilai_Kredit < 0;

PRINT 'Jika semua hasil Jumlah = 0 atau tabel kosong, berarti Kualitas Data
BAGUS (100%).';

```

Step 7 : Test Performance

```

USE DM_Keuangan_DW;
GO

-- Nyalakan pencatat waktu
SET STATISTICS TIME ON;

PRINT '=== MULAI TEST DATA 10.000 BARIS ===';

-- Jalankan Query Berat (Join 5 Tabel sekaligus)
SELECT
    w.Tahun,
    u>Nama_Fakultas,
    SUM(f.Nilai_Debit) AS Total_Debit
FROM dbo.Fact_Transaksi f
JOIN dbo.Dim_Waktu w ON f.ID_Waktu = w.ID_Waktu
JOIN dbo.Dim_Unit_Org u ON f.ID_Unit = u.ID_Unit
GROUP BY w.Tahun, u>Nama_Fakultas
ORDER BY w.Tahun;

PRINT '=== SELESAI ===';

```

```
-- Matikan pencatat waktu  
SET STATISTICS TIME OFF;
```